REGULAR EXPRESSIONS

Finding patterns in text with Python's re module

Dr. Prabu M

WHY REGEX

The problem with plain string methods

Task: check if a string looks like a valid email address.

```
def is_email(s):
    if s.count('@') != 1:
        return False
    user, domain = s.split('@')
    if not user:
        return False
    if '.' not in domain:
        return False
    # ...growing forever
```

Every new rule = another if-statement.

Regex lets you describe the **shape** of valid text in one line.

```
import re
re.match(
  r"^[\w.]+@[\w]+\.\w+$", s)
```

Today: read patterns, write patterns, use them to validate and extract text.

CORE SYNTAX · 1 OF 4

Matching characters

Symbol		Example		Meaning
`Literal`		`abc`		matches exactly a, then b, then c
`.`		`a.c`		any single character except newline
`\d`	`\D`	`\d+`	`\D+`	a digit / a non-digit
`\w`	`\W`	`\w+`	`\W+`	letter/digit/underscore / anything else
`\s`	`\S`	`\s+`	`\S+`	whitespace / non-whitespace

CORE SYNTAX · 2 OF 4

Quantifiers — how many times

Symbol	Example	Meaning
`*`	`ab*`	0 or more of the preceding character
`+`	`ab+`	1 or more of the preceding character
`?`	`ab?`	0 or 1 of the preceding character
`{n}`	`a{3}`	exactly n occurrences
`{n,}`	`a{2,}`	n or more occurrences
`{n,m}`	`a{2,4}`	between n and m occurrences

CORE SYNTAX · 3 OF 4

Anchors & boundaries — where it matches

Symbol	Example	Meaning
`^`	`^Hello`	start of the string
`$`	`world$`	end of the string
`\A` `\Z`	`\AHi` `bye\Z`	absolute start / absolute end of string
`\b`	`\bcat\b`	a word boundary
`\B`	`\Bcat`	position with NO word boundary

CORE SYNTAX · 4 OF 4

Grouping & character sets

Symbol	Example	Meaning
`[ ]`	`[aeiou]`	any one character inside the brackets
`[^ ]`	`[^0-9]`	any character NOT in the set
`-`	`[a-z]`	a range, used inside [ ]
`( )`	`(cat|dog)`	groups part of a pattern together
`|`	`cat|dog`	OR — matches the expression before or after

Deliberately skipped today: lookaheads/lookbehinds, non-greedy (?), named groups.*

PUTTING IT TOGETHER

A few worked examples

Regex	Example Input	Matches
`r'\d+'`	`"Age: 25"`	25
`r'\w+'`	`"Hello_123"`	Hello_123
`r'\s+'`	`"Hello World"`	the space between words
`r'[A-Z][a-z]+'`	`"Python"`	Python
`r'\d{10}'`	`"9876543210"`	9876543210
`r'^\d+$'`	`"12345"`	matches — the entire string is digits

THE TOOLKIT

Four functions cover almost everything

`re.match()`

Checks the START of the string only.

```
re.match(r"\d+", "123abc")
# matches "123"
```

`re.search()`

Finds the pattern ANYWHERE in the string.

```
re.search(r"\d+", "abc123")
# finds "123"
```

`re.findall()`

Returns ALL matches as a list.

```
re.findall(r"\d+", "a1 b22 c3")
# ['1', '22', '3']
```

`re.sub()`

Replaces matches with new text.

```
re.sub(r"\d+", "#", "a1 b22")
# 'a# b#'
```

THE TOOLKIT

re.findall() — Find all matches

```
import re

text = "Python 3 is better than Python 2"
result = re.findall(r'Python', text)
print(result)

# Output: ['Python', 'Python']
```

Finds all non-overlapping matches and returns them as a list.

THE TOOLKIT

re.compile() — Create a reusable pattern

```
import re

pattern = re.compile(r'\d+')
print(pattern.findall("Marks: 85, Age: 20"))

# Output: ['85', '20']
```

Compiles a regex into a pattern object that can be reused many times.

THE TOOLKIT

re.split() — Split using a pattern

```
import re

text = "Python,Java;C++"
print(re.split(r'[,;]', text))

# Output: ['Python', 'Java', 'C++']
```

Splits a string wherever the specified pattern is found.

THE TOOLKIT

re.sub() — Replace matches

```
import re

text = "My phone is 9876543210"
print(re.sub(r'\d', '*', text))

# Output: My phone is **********
```

Replaces all matching occurrences with a replacement string.

THE TOOLKIT

re.subn() — Replace + count

```
import re

text = "cat dog cat bird cat"
result = re.subn(r'cat', 'lion', text)
print(result)

# Output: ('lion dog lion bird lion', 3)
```

Same as re.sub(), but also returns the number of substitutions made.

THE TOOLKIT

re.escape() — Treat special characters literally

```
import re

pattern = re.escape("C++")
print(pattern)

# Output: C\+\+
```

Escapes regex-special characters so they are matched literally.

THE TOOLKIT

re.search() — Find first occurrence

```
import re

text = "My marks are 95"
match = re.search(r'\d+', text)
print(match.group())

# Output: 95
```

Searches for the first occurrence of a pattern anywhere in the string.

REMEMBER...

- `findall()` → all matches
- `search()` → first match
- `split()` → divide string
- `sub()` → replace
- `subn()` → replace + count
- `compile()` → reusable pattern
- `escape()` → treat characters literally

LIVE CODE #1

Validating an email address

```
import re

pattern = r"^[\w.]+@[\w]+\.\w+$"

def is_valid_email(s):
    return re.match(pattern, s) is not None

print(is_valid_email("prabu.cse@amrita.edu"))    # True
print(is_valid_email("not-an-email"))             # False
```

Break it down, piece by piece

`^[\w.]+` one or more letters/digits/dots, from the start

`@` a literal @ symbol

`[\w]+\.\w+$` domain, a dot, then the ending, to the end of string

LIVE CODE #2

Extracting phone numbers — spot the bug

```
text = "Call Raj at 9876543210 or the office at 044-28451234."
```

```
# First attempt — what's wrong here?
numbers = re.findall(r"\d+", text)
print(numbers)
# ['9876543210', '044', '28451234']
# the office number got split at the hyphen!
```

Fixed version — allow an optional hyphen inside the pattern

```
numbers = re.findall(r"\d{2,4}-?\d{6,8}", text)
print(numbers)
# ['9876543210', '044-28451234']

# -?  means the hyphen is optional
```

YOUR TURN — SOLO, 10 MINUTES

Write these two patterns yourself

1. Validate a PIN code

Write a pattern matching a 6-digit Indian PIN code — exactly 6 digits, nothing else.

```
test("600036")   # True
test("6000366")  # False
test("60003a")   # False
```

Hint: ^ and $ anchor the whole string.

2. Extract hashtags

Pull every hashtag out of a sentence using re.findall().

```
s = "Loving the #python and
     #regex session today!"
# ['#python', '#regex']
```

Hint: a hashtag is # followed by one-or-more word characters.

2-question exit ticket

Q1

Write a regex pattern that matches a string starting with "PY" followed by exactly 3 digits.

Q2

What will `re.findall(r"\d+", "a12 b3c456")` return?

Next class: Multithreading + Thread Essentials